

SOFTWARE DESIGN DOCUMENT

(SDD)

*Prepared in the format of IEEE Std 1016-2009, Software Design Descriptions, and adapted to the
ENGGSF1/ENGGSF2 (Software Engineering 1 & 2) course sequence*

Project Title:

Kody: Gamified Programming Learning Platform and Course Management System

Course: ENGGSF2 - Software Engineering 2

Prepared by:

*De Guzman, Asher Nathaniel E., 20244549@s.ubaguio.edu
Dela Cruz, Rupert C., 20244042@s.ubaguio.edu
Macaraeg, Mark Cyrus P., 20244548@s.ubaguio.edu
Zapanta, Zeus Levi C., 20240821@s.ubaguio.edu*

Adviser: Martinez, Erna-kristi N.

Date: September 19 2026

Table of Contents

Table of Contents.....	1
1. Introduction.....	2
1.1 Purpose.....	2
1.2 Scope.....	2
1.3 Definitions, Acronyms, and Abbreviations.....	2
1.4 Document Overview.....	5
2. References.....	6
3. Architectural Design.....	6
3.1 Logical View.....	6
3.2 Physical View.....	7
3.3 System Architecture.....	8
4. Decomposition Description.....	10
4.1 Module Decomposition.....	11
4.2 Process Description.....	12

1. Introduction

1.1 Purpose

This Software Design Document (SDD) translates the approved Kody Software Requirements Specification (SRS), Version 1.4, into a design for implementation, integration, testing, and maintenance (Tsui et al., 2022). It defines architectural boundaries, module responsibilities, deployment targets, and coordination rules using the course-adapted structure of IEEE Std 1016-2009 (Institute of Electrical and Electronics Engineers [IEEE], 2009). Its intended readers are the project team and project manager, current and future developers and maintainers, quality-assurance personnel, the course instructor and adviser, and academic reviewers or panel members. They shall use it to implement the system consistently and assess whether design decisions satisfy approved requirements.

1.2 Scope

Kody: Gamified Programming Learning Platform and Course Management System is a web platform that combines structured programming courses, coding practice, automated evaluation, progress tracking, and deterministic rewards. This SDD covers Account and Authentication Management, Content Management, Coding Challenge Management, Gamification and Rewards, User Interaction and Engagement, Transaction and Earnings Management, and Administration and Governance (De Guzman et al., 2026). It includes token-based premium access, publisher earnings, moderation, and the approved external-service integrations. By connecting learning content, practice, and feedback, the project aims to support quality education (SDG 4). Version 1 excludes full institutional learning-management and academic-administration functions, non-programming subject domains, and chance-based reward mechanisms.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym/Abbreviations	Definition
API	Application Programming Interface: operations exposed to other software.
Asynchronous processing	Allows work to finish independently of the initiating request.
AWS	Amazon Web Services: cloud provider.
Business logic	Enforces system rules.
Client-server; layered architecture	Client-server separates browser requests from server processing. Layered architecture separates presentation, business rules, and data access.

Term / Acronym/Abbreviations	Definition
Commit; rollback	Commit makes a transaction's changes persistent. Rollback cancels its uncommitted changes.
Concurrency	Allows operations to overlap.
Data access	Reads and writes persistent records.
Data dictionary	A data dictionary defines fields.
Database transaction; atomic operation	A transaction groups related changes. Atomic means all changes succeed together or none take effect.
Deterministic reward	A deterministic reward follows predefined conditions rather than chance.
EC2 xx.xxx	Elastic Compute Cloud: application-server hosting. May be followed by the plan/subscription type, eg. t2.small.
ERD	Entity-Relationship Diagram: shows data entities and relationships.
FAQ	Frequently Asked Questions: published answers to common user questions.
Gamification	Gamification adds progression and reward features to learning.
GB	Gigabyte.
Google Identity	Google Identity is the external identity service; Kody manages its own roles.
HCI	Human-Computer Interaction: design of interaction with computers.
Horizontal scaling	Horizontal scaling adds instances; application and database scaling require separate designs.
HTTPS	Hypertext Transfer Protocol Secure: protected web communication.
Idempotency; callback; webhook	Idempotency prevents repeated requests from repeating effects. A callback reports a result; a webhook sends an event to a server endpoint.
Judge0	External services for isolated code execution.

Term / Acronym/Abbreviations	Definition
KodeBits	KodeBits are non-transferable platform tokens within Kody
Learning Module	A self-contained, structured unit of instruction designed to teach a specific topic or skill
LMS	Learning Management System: software for managing learning delivery.
Microservices	Microservices deploy services independently.
Modular monolith	A modular monolith deploys cooperating modules as one application.
Module	Module, as used in this document, refers to a component with a defined responsibility. Do not mistaken for “Learning Module”
OAuth 2.0	OAuth 2.0 is an authorization framework used in the sign-in integration.
Persistence; backup; fault tolerance	Persistence retains data after a request or restart. A backup supports recovery. Fault tolerance limits disruption when a component fails.
PostgreSQL	PostgreSQL: the selected relational database system.
Premium access	Premium access uses tokens to unlock content.
Presentation layer	Displays information and receives input.
Publisher earnings	Publisher earnings arise from validated content transactions.
Rate limit	Restricts request frequency.
RDS xx.xxx	Relational Database Service: managed database hosting. May be followed by the plan/subscription type, eg. t3.micro
Record lock; record version	A lock coordinates competing changes to a record. A version identifies whether a record changed after it was read.
Repository	A repository encapsulates access to stored application records.

Term / Acronym/Abbreviations	Definition
REST	Representational State Transfer: an approach to designing web interfaces around resources.
RTM	Requirements Traceability Matrix: links requirements to design and verification.
S3	Simple Storage Service: object storage.
Sandbox	Isolates submitted code.
SendGrid	External services for transactional email delivery.
SQL	Structured Query Language: language for working with relational data.
Static asset	A static asset is a file such as an image.
Test case	Supplies input and an expected result.
TLS	Transport Layer Security: the protocol securing data in transit.
UI	User Interface: visible controls and screens.
UX	User Experience: the overall experience of using the system.
Vertical scaling	Vertical scaling increases an existing resource's capacity.
virtual CPU	A virtual CPU is an allocated processor unit.
Xendit	External services for payment processing.

1.4 Document Overview

Section 2 identifies the sources supporting the design. Section 3 presents the logical architecture, physical deployment, and request-processing structure. Section 4 decomposes the system into modules and explains how concurrent processes coordinate. Sections 5 and 6 provide the planned dependency and interface descriptions. Section 7 covers the planned entity-relationship diagram and data dictionary, while Section 8 covers component algorithms and human-computer interaction designs. Section 9 provides the planned requirements traceability matrix, and Appendix A records document revisions. Sections 1–4 form the current first-grading deliverable; the later sections remain reserved for subsequent development.

2. References

Cha, S., Taylor, R. N., & Kang, K. (2019). Handbook of Software Engineering. Springer.

De Guzman, A. N. E., Dela Cruz, R. C., & Macaraeg, M. C. P. (2026). *Software requirements specification for Kody: Gamified programming learning platform and course management system* (Version 1.4) [Unpublished requirements specification]. University of Baguio.

Institute of Electrical and Electronics Engineers. (2009). *IEEE standard for information technology—Systems design—Software design descriptions* (IEEE Std 1016-2009).

Tilley, S. (2019b). Systems analysis and design. Course Technology.

Tockey, S. (2019). How to Engineer Software: A Model-Based Approach. John Wiley & Sons.

Tsui, F., Karam, O., & Bernal, B. (2022b). Essentials of Software Engineering (5th ed., Vols. 7–10). Jones & Bartlett Learning

3. Architectural Design

3.1 Logical View.

Kody shall follow a layered client-server modular monolith architecture with event-driven or asynchronous integration. The client-server structure separates the browser-based user interface from server-side processing, while the modular monolith organizes the system's functionality into independent internal modules within a single application deployment. The architecture is divided into three (3) primary layers:

- **Presentation Layer:** provides the web interface used by Learners, Contributors, Instructors, Moderators, and Administrators.
- **Application and Business Logic Layer:** contains the seven functional modules of Kody: Account and Authentication Management, Content Management, Coding Challenge Management, Gamification and Rewards, User Interaction and Engagement, Transaction and Earnings Management, and Administration and Governance.
- **Data Access Layer:** manages persistent application data and communication with database and storage resources.

External services (Google Identity, Judge0, SendGrid, and Xendit) are accessed through defined service interfaces rather than being embedded directly into individual modules. The SRS requires these integrations to use well-defined APIs so that external providers can later be replaced or upgraded without major redesign.

Notifications and leaderboard updates shall run asynchronously so they do not block normal user interaction (De Guzman et al., 2026, Section 4.2). For the four-member student team, a modular monolith reduces deployment and integration overhead within the academic schedule while keeping responsibilities separable for maintenance. Its trade-off is that modules share application resources and a deployment lifecycle; a failure can affect several functions. Figure 1.0 shows the layers, seven modules, and external-service boundary. Background processing belongs to the Application and Business Logic Layer.

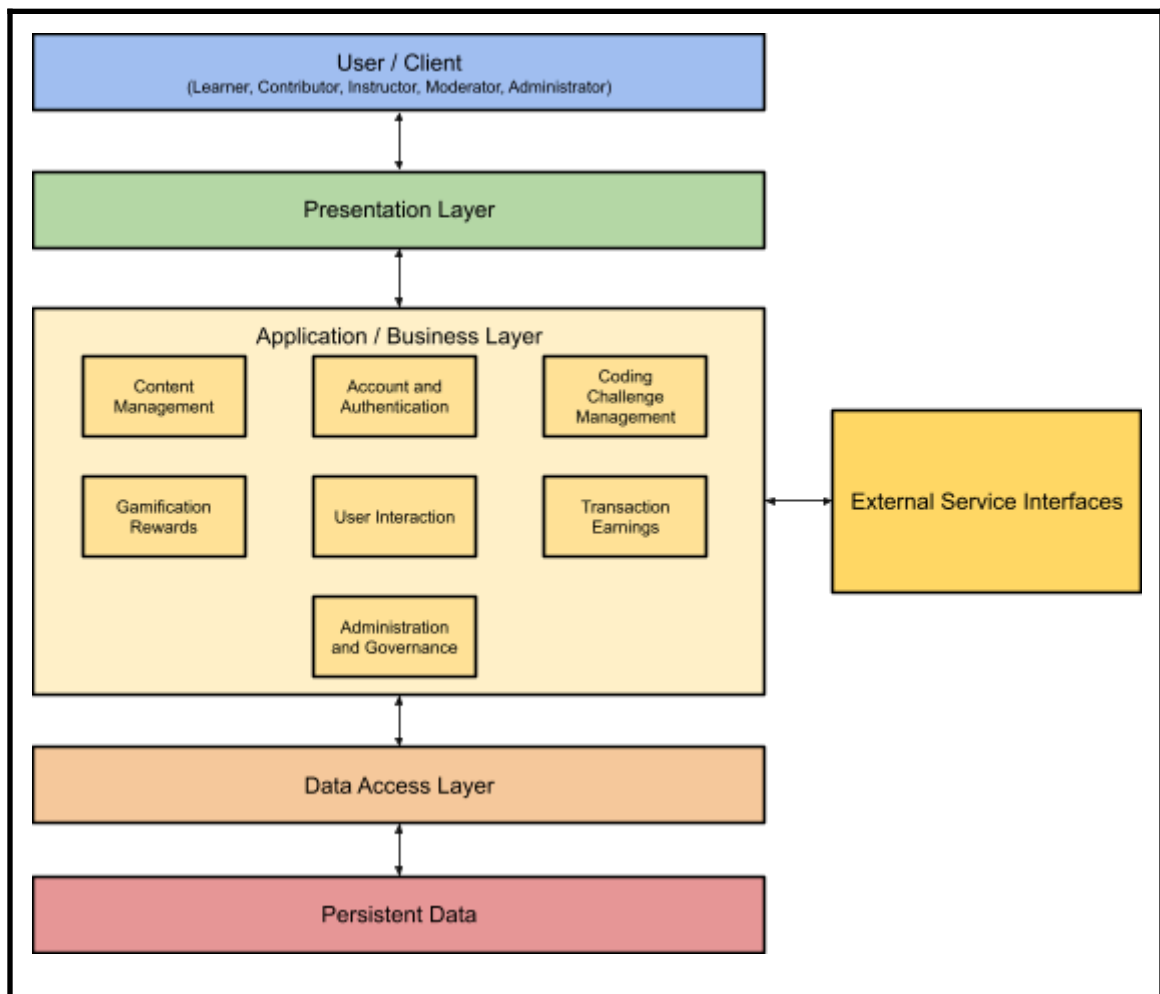


Figure 1.0 - Logical Architecture Diagram.

3.2 Physical View.

Figure 2.0 maps the logical design to its deployment targets. The Presentation Layer runs in desktop or mobile browsers. AWS EC2 hosts all seven business modules, the Data Access Layer, external-service interfaces, scheduled tasks, and background workers. AWS RDS stores relational application data using PostgreSQL; Amazon S3 (Simple Storage Service) stores static assets. This mapping follows the SRS cloud-hosting requirements (De Guzman et al., 2026, Section 3.1).

Google Identity handles external sign-in, Judge0 executes submitted code in isolation, SendGrid delivers email, and Xendit processes payments. The external-service connections shown at the AWS boundary originate from the application on EC2; they do not give providers direct database access. The “OAuth 2.0” label identifies the Google Identity integration and its authorization protocol.

The SRS initial configuration is EC2 t3.small (2 virtual CPUs and 2 GB memory plan) and RDS db.t3.micro. A single application deployment reduces deployment and coordination work for the four-member student team. Managed database and storage services separate persistent resources from application processing, but introduce provider costs, network latency, and dependence on external availability.

The centralized application is a shared failure point and has limited capacity. Application instances and workers may be added as load grows, with coordinated jobs and shared persistent records. Database scaling requires separate capacity planning; adding application instances alone does not scale database writes. Load and recovery testing shall determine whether the deployment meets the SRS performance and availability targets.

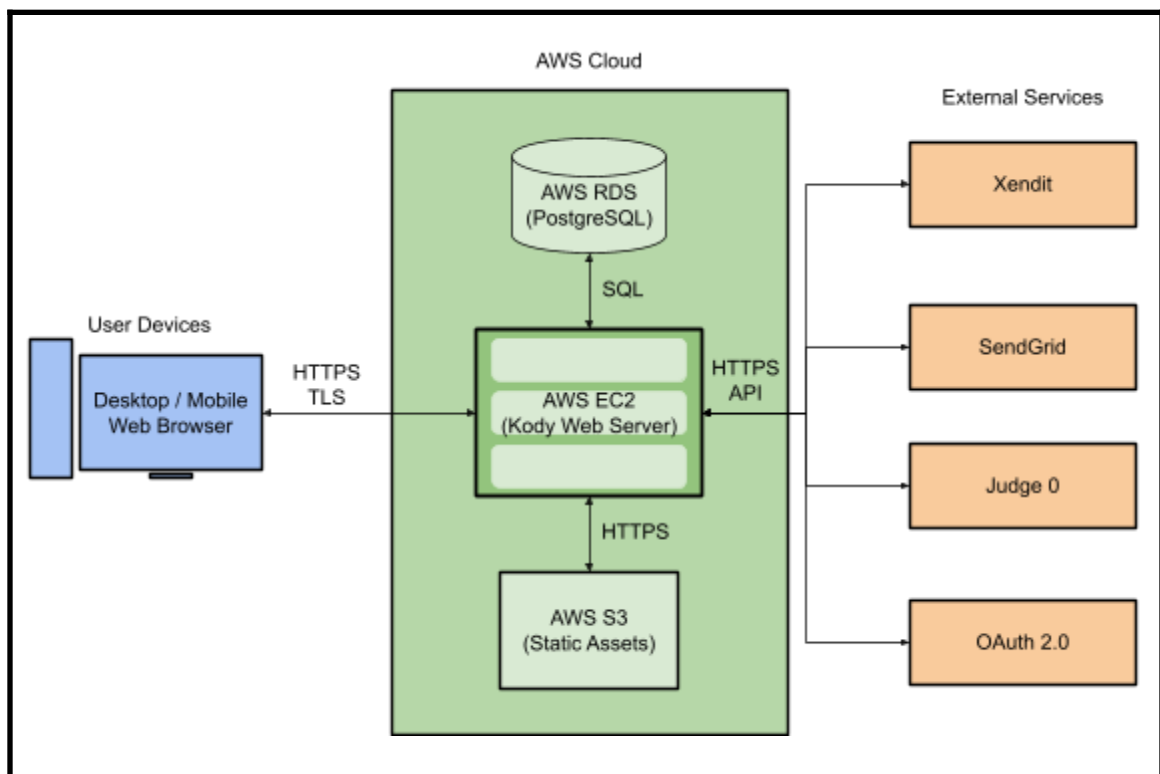


Figure 2.0 - Physical Deployment Diagram.

3.3 System Architecture.

Kody shall implement a layered client-server modular monolith architecture (Tilley, 2019; Tockey, 2019; Tsui et al., 2022). Its seven functional modules shall operate within a single application deployment, with defined responsibilities and internal

interfaces. Shared facilities shall support authentication checks, data access, external service integration, logging, and background processing.

A user action shall begin at the browser interface and be sent to the application server over HTTPS. The server shall validate the request, check the user's session and their permissions, and route the operation to the responsible module. That module shall apply the relevant business rules and access persistent records through the Data Access Layer. Operations involving multiple related records, such as deducting KodeBits and granting content access, shall use a database transaction so that the changes either succeed together or are rolled back.

Modules shall coordinate through internal service calls for operations requiring an immediate result and through background jobs for asynchronous work. Notifications and leaderboard updates shall run independently of the initiating browser request. Pending jobs shall be stored persistently, allowing interrupted processing to resume without losing the associated operation.

External services shall be accessed through dedicated integration interfaces: Google Identity for authentication, Judge0 for isolated code execution, SendGrid for email delivery, and Xendit for payment processing. The application shall validate external responses before updating its records. User-submitted code shall execute in the isolated execution environment and shall have no access to Kody's application database or internal services. Using defined interfaces between the application and external services supports modularity and reduces dependency between internal components and specific external implementations (Tockey, 2019; Cha et al., 2019).

This architecture supports the SRS requirements for modular maintenance, asynchronous processing, secure integrations, and persistent data. The required 500 concurrent users, 3–5-second code feedback, and updates within 2 seconds of validated activity are acceptance targets to be verified through load testing; the initial deployment does not itself prove these targets are met (De Guzman et al., 2026, Sections 3.4 and 4.2).

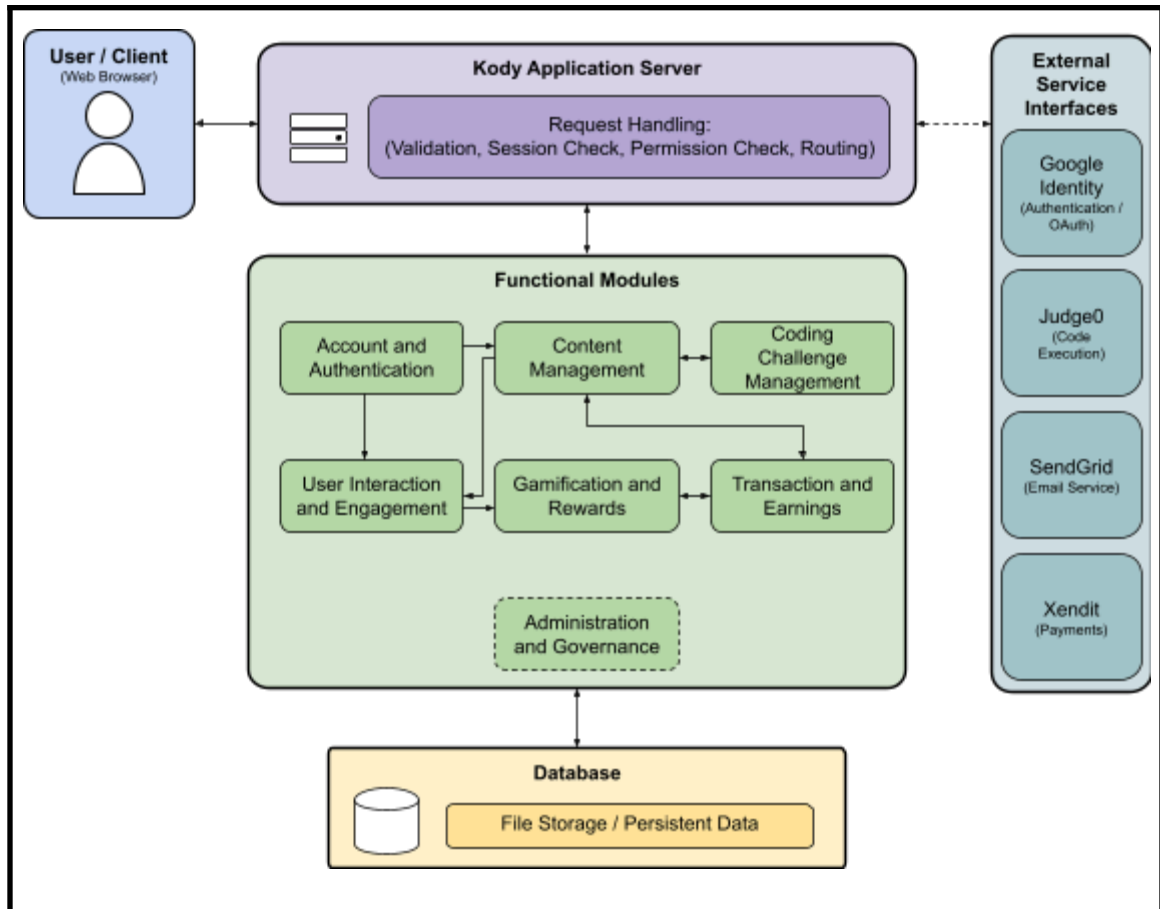


Figure 3.0 - System Architecture of Kody.

4. Decomposition Description

Kody is decomposed into seven functional modules corresponding to the capabilities defined in the SRS. Each module owns a specific set of responsibilities while coordinating with other modules through controlled interfaces. These are logical divisions within the application; they do not require separate servers or independently deployed services. Table 1.0 lists their responsibilities, sub-functions, and SRS use-case references.

4.1 Module Decomposition

Module Name	Responsibility	Key Sub-Functions
Account and Authentication Management	Manage user identity, authentication, account lifecycle, and role-request records.	Register accounts and verify email; authenticate users and recover accounts; view, edit, archive, and delete accounts; submit contributor requests and support instructor credential verification. SRS: A01–A10.
Content Management	Manage the structure and lifecycle of courses and learning modules.	Create, edit, archive, and delete modules; create, edit, archive, and delete courses; assign modules to courses. Publication shall respect the approval requirements enforced through Administration and Governance. SRS: D01–D09; G06.
Coding Challenge Management	Manage coding challenges, submissions, automated evaluation, and stored feedback.	Create and approve challenges; edit, archive, and delete challenges; validate and record submissions; enforce submission limits; coordinate sandboxed evaluation; store scores, execution metrics, and feedback. SRS: C01–C07.
Gamification and Rewards	Apply deterministic game rules, reward conditions, and ranking calculations.	Generate activities from presets; configure weekly challenges; aggregate weekly evaluation results; publish weekly results; grant eligible rewards; determine rankings. SRS: E01–E06.
User Interaction and Engagement	Coordinate learner-facing access to content, progress information, and participation features.	Display dashboards; browse courses, modules, and challenges; enroll in courses; access course and standalone modules; participate in challenges; display execution feedback and leaderboards; react to content; view FAQs. SRS: B01–B11.

Module Name	Responsibility	Key Sub-Functions
Transaction and Earnings Management	Maintain KodeBits transactions, premium-access purchases, publisher earnings, and event-based notifications.	Purchase and spend tokens; verify payment confirmations; record balance changes and access grants; display publisher earnings; process instructor payouts and contributor KodeBits credits; deliver in-app and email notifications. SRS: F01–F05.
Administration and Governance	Enforce platform rules and manage privileged administrative functions.	View and update user accounts; suspend and reinstate accounts; approve contributor requests; moderate content; view system reports; create, update, and delete game presets and FAQs. SRS: G01–G13.

Table 1.0 - Module Decomposition and Traceability to SRS Section 4.1 (De Guzman et al., 2026).

Where responsibilities meet, one module shall retain ownership of the underlying operation. For example, User Interaction and Engagement shall request challenge submission from Coding Challenge Management, while Gamification and Rewards shall request KodeBits credits through Transaction and Earnings Management. This prevents separate modules from applying conflicting submission or balance rules.

4.2 Process Description

Kody shall handle independent user requests concurrently while coordinating operations that share records. Database transactions, record versions, unique operation identifiers, and persistent background jobs shall protect committed data. The following mechanisms implement the concurrency and asynchronous-processing requirements in SRS Section 4.2 (De Guzman et al., 2026).

Payments and access. Transaction and Earnings Management shall commit the balance check, token deduction, transaction record, and content-access grant together. A lock on the affected balance record shall prevent simultaneous spending from producing a negative balance. Verified Xendit callbacks shall be matched to unique transaction identifiers so repeated callbacks cannot credit tokens twice. Pending instructor payouts shall reserve the requested amount against further requests; the recorded earnings deduction shall occur only after verified payout success (F01–F04).

Submissions and evaluation. Coding Challenge Management shall store submissions before evaluation and update attempt counts as the same record, preventing simultaneous requests from exceeding the three-submission limit. Each evaluation job shall be claimed by one worker and matched to its submission identifier. Evaluation shall respect C04's confirmation and interruption rules, and stored results shall remain retrievable after processing (C03–C04; B08).

Weekly activities, rewards, and rankings. Scheduled and moderator-triggered operations shall use the same weekly-instance record and activation lock, preventing two active weekly challenges. Aggregation shall begin only after the window closes and the required evaluations finish. Unique reward records shall prevent duplicate grants when events are repeated. Leaderboard processing shall publish a completed result set together, retaining the previous committed results if computation fails (E02–E06; B09).

Content and account changes. Content, challenge, and governance operations shall check record versions before saving. An outdated edit shall be rejected with instructions to reload and review the current record. Privileged changes shall recheck the acting user's authorization and the target record's status before committing, preventing conflicting approval, suspension, or publication actions (A06; C02, C05–C07; D01–D09; G02–G06).

Background processing and recovery. Notifications and leaderboard updates shall run independently of browser requests. The initiating change and its pending job shall be stored together; workers shall claim jobs exclusively and record completion. Failed notifications shall use controlled retries without reversing a completed purchase or reward. Repeated jobs shall recognize completed operations, and returning users shall see the last committed state (E05–E06; F05; SRS Section 4.2).